GUIDED LLM PROMPT

NC STATE Engineering

Recreate the Full ML Pipeline Notebook

A classroom-ready prompt sequence for using an LLM to generate the same structure, code, and interpretation as the Day 3 notebook workflow.

Prompt chain outputs

- notebook sections
- Python code cells
- metrics + plots
- model card summary

Applied AI Design Workshop for Faculty
Fred Livingston • 2026

Activity framing

Why use a guided LLM prompt?

The goal is not to let the LLM “do the science.” The goal is to teach how to use an LLM to produce a reproducible notebook scaffold that still requires human validation.

LLM is useful for

- Generating notebook structure
- Explaining code line by line
- Drafting scikit-learn workflows
- Writing interpretation templates

Human must validate

- Target and unit of observation
- Leakage and data availability
- Metric choice
- Unsupported claims or causal language

Takeaway

AI accelerates the pipeline, but evidence, assumptions, and conclusions remain the researcher’s responsibility.

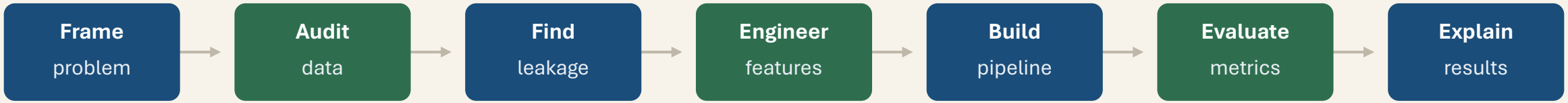
Use the prompt chain as a transparent recipe: each prompt produces one part of the notebook, and each output is checked before the next step.

Guided sequence

7 PROMPTS

Prompt chain overview

Each prompt should return a **notebook section**, suggested code, and a checklist for human verification.



What should be generated?

- Markdown explanation for the notebook
- Executable Python code cells
- Expected outputs or interpretation cues
- A human-check question before continuing

What should not be delegated?

- Approving labels or assumptions
- Deciding whether unusual values are real
- Making causal claims from predictive models
- Declaring deployment readiness

Prompt 0

GUARDRAILS

Setup prompt: define the LLM's role and boundaries

COPY-PASTE PROMPT 0

```
You are helping create a Jupyter notebook for an applied AI workshop.

Use Python, pandas, matplotlib/seaborn, and scikit-learn.
Create short markdown explanations and executable code cells.

Do not invent dataset columns. Ask me to provide column names if needed.
Do not claim causality from prediction results.
Do not remove outliers without a documented domain-based rule.
Preprocessing must be fitted only on training data.
```

Note: before running the prompt chain, provide the dataset name and columns. This prevents the LLM from guessing the data structure.

Step 1

Prompt 1: problem framing

COPY-PASTE PROMPT 1

```
Create the first notebook section for this dataset:

Dataset: classification_workshop.csv
Columns: temperature_c, pressure_kpa, speed_mm_s,
vibration_mm_s, defect

Write markdown and code that identify:
- the unit of observation
- input variables and units
- target variable and target meaning
- model task
- baseline
- primary metric

Return the result as notebook-ready markdown and Python
code.
```

Expected result

A notebook section that clearly states: one row = one process run; inputs are process measurements; target = defect; task = binary classification.

Human check

Are these variables available before the prediction would be made? Are the units and target definition correct?

Classroom question

If the target were continuous yield instead of defect, how would the task and metric change?

Step 2

Prompt 2: data audit and visualization

COPY-PASTE PROMPT 2

```
Generate notebook cells to load
classification_workshop.csv and audit it.

Include code for:
- df.head(), df.shape, df.dtypes
- missing values
- duplicate rows
- target class balance
- summary statistics
- a count plot of defect
- scatter or box plots for process variables

Add brief markdown that explains what each audit output
tells us.
```

Useful output is not just code; it should tell participants what to look for in the results.

Evidence to inspect

- class imbalance
- missing values
- unexpected data types
- range or scale differences
- variables that may be identifiers

Human check

Do the plots support the proposed task, or do they reveal that the data are too limited or noisy for the desired claim?

Step 3

Prompt 3: leakage and availability review

COPY-PASTE PROMPT 3

```
Review these candidate inputs for target leakage.

Prediction time: before final quality inspection.
Inputs: temperature_c, pressure_kpa, speed_mm_s,
vibration_mm_s.
Target: defect.

Create a notebook markdown table with columns:
variable, role, available at prediction time?, leakage
risk?, decision.

Add one paragraph explaining why leakage can invalidate
model evaluation.
```

What the LLM can do

Flag variables that might be unavailable or outcome-derived; suggest questions to ask the data owner.

What the LLM cannot know

Whether a sensor value was recorded before, during, or after the decision unless you provide that temporal context.

Emphasis

Leakage is a workflow problem, not just a coding problem. The timeline defines what the model is allowed to know.

Step 4

Prompt 4: feature engineering with discipline checks

COPY-PASTE PROMPT 4

```
Suggest candidate engineered features for a defect-
classification notebook using:
temperature_c, pressure_kpa, speed_mm_s, vibration_mm_s.

Separate the suggestions into:
1. domain-informed features
2. mathematically convenient features
3. features that require caution

Provide Python code for only the clearly justified
features.
Do not claim that any feature causes defects.
```

Example outputs

- temperature deviation from nominal
- pressure deviation from target
- vibration-to-speed ratio
- process intensity proxy

Verification questions

- Is the nominal value known?
- Does the feature have a physical meaning?
- Is it available before prediction?
- Does it duplicate or leak the target?

Step 5

CODE CELLS

Prompt 5: generate the reproducible notebook workflow

COPY-PASTE PROMPT 5

```
Create the next notebook section using scikit-learn.

Build a reproducible binary-classification workflow that:
- defines X and y
- uses train_test_split with stratify=y and random_state=42
- creates a majority-class baseline
- standardizes numeric features
- fits LogisticRegression(max_iter=1000)
- predicts on the test set only
- reports accuracy, precision, recall, F1, and confusion matrix

Use clear markdown headings and short explanations before each code cell.
```

Prompt after generation: “Where is preprocessing fitted? Did the code evaluate on the held-out test set? Is the baseline included?”

Step 6

NC STATE Engineering METRICS

Prompt 6: evaluation and error analysis

COPY-PASTE PROMPT 6

```
Add a notebook section that interprets the model results.

Use these concepts:
- baseline comparison
- confusion matrix
- false positives and false negatives
- accuracy versus recall
- precision, recall, and F1

Write markdown prompts that ask the participant:
1. Did the model beat the baseline?
2. Which error is more costly?
3. Is the improvement practically meaningful?
```

Expected notebook output

A table comparing baseline and model metrics plus a confusion matrix with labels meaningful to the application.

Evaluation emphasis

Accuracy can improve while recall remains too low for quality inspection or safety-sensitive use.

Ask participants to select the metric before viewing model performance.

Step 7

NC STATE Engineering DOCUMENTATION

Prompt 7: interpretation and model card

COPY-PASTE PROMPT 7

```
Create the final notebook section: an initial model card.

Include fields for:
- intended use
- out-of-scope use
- dataset and target
- baseline result
- model result
- primary error pattern
- supported conclusion
- unsupported conclusion
- limitations
- recommended next analysis

Avoid causal claims and deployment-ready language.
```

Strong interpretation

"The model improves over baseline but misses many actual defects, so it may support review prioritization only after further validation."

Weak interpretation

"The model proves which process settings cause defects and is ready for production use."

Output

One concise evidence-grounded paragraph that could be reused in a report, proposal, or class example.

Combined prompt

NC STATE Engineering ADVANCED

One-shot prompt: generate the full notebook scaffold

COPY-PASTE FULL PROMPT

```
Create a complete Jupyter notebook for an applied AI workshop using classification_workshop.csv.
Columns: temperature_c, pressure_kpa, speed_mm_s, vibration_mm_s, defect.

Notebook sections must include:
1. problem framing and unit of observation
2. data loading and audit
3. class balance and visualization
4. leakage review
5. X/y definition and stratified train/test split
6. majority-class baseline
7. standardized logistic regression workflow
8. accuracy, precision, recall, F1, and confusion matrix
9. interpretation, limitations, and a short model card

Use markdown plus executable code cells. Do not invent columns or causal claims.
```

Use the full prompt only after participants understand the seven-step chain. Prompt chaining is usually easier to debug and teach.

Live demo

NC STATE Engineering AI AUDIT

Class demonstration: before and after grounding

Ungrounded prompt

“Build a machine-learning notebook for defect prediction and explain the results.”

Risk: invented columns, unsupported claims, missing baseline.

Grounded prompt

Provides dataset name, exact columns, task, constraints, metrics, and required interpretation boundaries.

Class audit questions

- Did it invent any column names?
- Did it include a baseline?
- Did it fit preprocessing before the split?
- Did it overclaim causality?
- Did it produce a notebook section participants can run?

Instructor debrief

The LLM output becomes a draft artifact. The class evaluates the draft against the ML workflow, not against how polished the response sounds.

Participant activity

GROUP WORK

Prompt-chain worksheet

Each group uses the prompt sequence to generate one notebook section, then audits it before moving to the next step.

Group A

Problem framing
Data audit
Visualization

Check: task, target, units, class balance

Group B

Leakage review
Feature ideas
Pipeline code

Check: availability, preprocessing, split

Group C

Metrics
Interpretation
Model card

Check: baseline, error costs, limitations

Share-out format

- One useful LLM output
- One output that required correction
- One decision that required human expertise

Quality criteria

CHECKLIST

What the LLM should produce — and what it should not

Acceptable output

- clear notebook sections
- executable code
- explicit X/y definition
- baseline comparison
- held-out evaluation
- confusion matrix or metrics
- cautious interpretation
- limitation statement

Reject or revise output

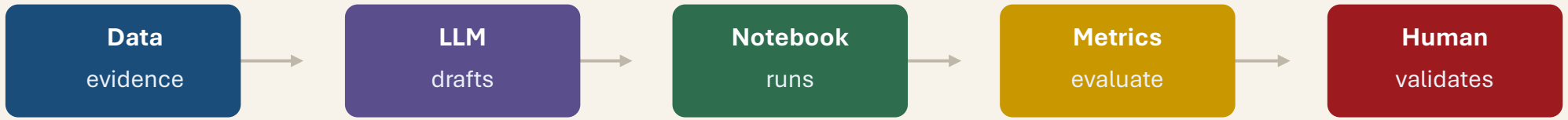
- invented variables
- no baseline
- no train/test split
- preprocessing on full data before split
- accuracy-only conclusion
- causal claims
- deployment-ready language
- missing human verification questions

Evaluation rule: a prompt-generated notebook is useful only if you can inspect, run, and defend every step.

Takeaways

Closing synthesis

The prompt chain converts the Day 3 pipeline into an AI-assisted workflow while preserving the core responsibilities of applied machine learning.



Final message to participants

AI can accelerate every stage of the ML pipeline, but it does not remove the need to define the task, check data quality, prevent leakage, evaluate models, and state limitations.